Sample slides

Examples and code testing

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

Why Git (and what you actually need)

Git is a *distributed version control system*. You use it to:

- track changes in code,
- collaborate without overwriting each other,
- recover from mistakes (yes, including stupid ones).

Mental model:

- **Repository**: the project + its history.
- **Commit**: a snapshot with a message (“what changed and why”).
- **Remote**: shared version (e.g. GitLab/GitHub).

You do *not* need:

- rebasing wizardry,
- cherry-picking,
- Git archaeology.

The only Git workflow you need (for this course)

Basic collaboration pattern:

1. Clone the repository
2. Create your own branch
3. Make changes + commit
4. Push branch
5. Merge via main branch

The only Git workflow you need (for this course)

Basic collaboration pattern:

1. Clone the repository
2. Create your own branch
3. Make changes + commit
4. Push branch
5. Merge via main branch

Essential commands:

```
git clone <repo-url>
git checkout -b my-branch
git status
git add file.py
git commit -m "Clear message"
git push origin my-branch
```

The only Git workflow you need (for this course)

Basic collaboration pattern:

1. Clone the repository
2. Create your own branch
3. Make changes + commit
4. Push branch
5. Merge via main branch

Essential commands:

```
git clone <repo-url>
git checkout -b my-branch
git status
git add file.py
git commit -m "Clear message"
git push origin my-branch
```

Rules that save friendships:

- Never work directly on `main`
- Commit often, commit small
- If something breaks: Git remembers, you don't have to

What Git does NOT solve for you

Important realism check:

- Git does not fix bad code
- Git does not prevent bad ideas
- Git does not write commit messages for you

Merge conflicts are not errors — they are Git saying:

> *“Two people changed the same thing. You decide.”*

If you fear conflicts, you waited too long to commit.

Exercise: first collaborative Git workflow

Setup (teacher):

- Create a practice repository with:
  - `README.md`
  - `compute.py` (simple placeholder function)
- Add students as collaborators

Exercise: first collaborative Git workflow

Setup (teacher):

- Create a practice repository with:
  - `README.md`
  - `compute.py` (simple placeholder function)
- Add students as collaborators

Student task:

1. Clone the repository
2. Create a branch named `firstname-lastname`
3. Modify `compute.py`:
   - add a small function (e.g. square, cube, norm, etc.)
   - add your name in a comment
4. Commit with message:

   "Add <function-name> by <name>"
5. Push your branch

Exercise: first collaborative Git workflow

Setup (teacher):

- Create a practice repository with:
  - `README.md`
  - `compute.py` (simple placeholder function)
- Add students as collaborators

Student task:

1. Clone the repository
2. Create a branch named `firstname-lastname`
3. Modify `compute.py`:
   - add a small function (e.g. square, cube, norm, etc.)
   - add your name in a comment
4. Commit with message:

   "Add <function-name> by <name>"
5. Push your branch

Bonus (controlled conflict):

- Two students edit the same line in `README.md`
- Merge branches and resolve the conflict together